AI ASSISTED CODING

Assignment-14.5

Name:E.THARUN

Batch:36

Ht.No:2303A52352

Task 1 – Image Gallery Website

Create an image gallery web page.

Requirements:

• Display images in a grid layout.

• Use AI to:

o Optimize layout using CSS Grid.

o Add hover zoom or overlay effects.

o Ensure mobile responsiveness.

Image Gallery HTML Structure

This HTML code defines the basic structure of the image gallery, including a header, a main gallery container, and several image items with placeholder images.

[11]
0s

```
html_code = """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI-Generated Image Gallery</title>
    <link rel="stylesheet" href="style.css"> <!-- Link to our CSS file -->
</head>
<body>
    <header>
        <h1>My Awesome Image Gallery</h1>
    </header>
    <main class="gallery-container">
        <div class="gallery-item">
            <img src="https://picsum.photos/id/10/300/200" alt="Gallery Image 1">
            <div class="overlay"><span>Image Title 1</span></div>
        </div>
        <div class="gallery-item">
            <img src="https://picsum.photos/id/20/300/200" alt="Gallery Image 2">
            <div class="overlay"><span>Image Title 2</span></div>
```

```
        <div class="gallery-item">
            <img src="https://picsum.photos/id/40/300/200" alt="Gallery Image 4">
            <div class="overlay"><span>Image Title 4</span></div>
        </div>
        <div class="gallery-item">
            <img src="https://picsum.photos/id/50/300/200" alt="Gallery Image 5">
            <div class="overlay"><span>Image Title 5</span></div>
        </div>
        <div class="gallery-item">
            <img src="https://picsum.photos/id/60/300/200" alt="Gallery Image 6">
            <div class="overlay"><span>Image Title 6</span></div>
        </div>
        <div class="gallery-item">
            <img src="https://picsum.photos/id/70/300/200" alt="Gallery Image 7">
            <div class="overlay"><span>Image Title 7</span></div>
        </div>
        <div class="gallery-item">
            <img src="https://picsum.photos/id/80/300/200" alt="Gallery Image 8">
            <div class="overlay"><span>Image Title 8</span></div>
        </div>
        <div class="gallery-item">
            <img src="https://picsum.photos/id/90/300/200" alt="Gallery Image 9">
            <div class="overlay"><span>Image Title 9</span></div>
        </div>
    </main>
</body>
</html>
"""
```

```
"""

with open("index.html", "w") as f:
    f.write(html_code)

print("HTML code saved to index.html")
```

```
HTML code saved to index.html
```

HTML code saved to index.html

Image Gallery CSS Styling

This CSS code styles the gallery, implementing a responsive CSS Grid layout, hover zoom effects, and overlay effects for image titles. It ensures a clean and modern look.

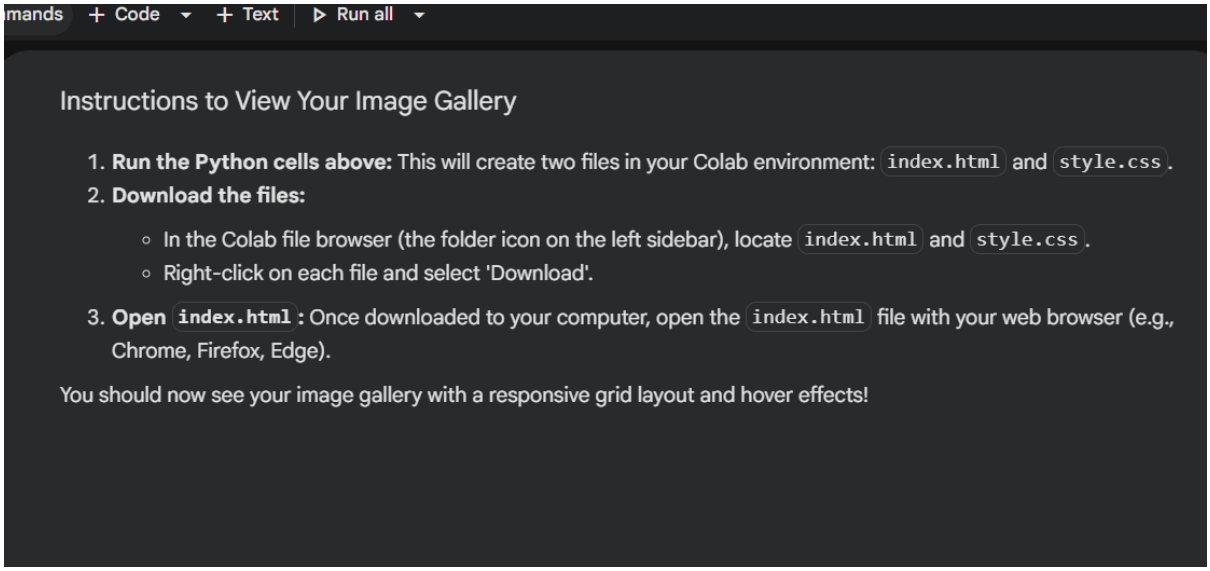
[12]
0s

```
css_code = """
body {
    font-family: Arial, sans-serif;
    margin: 0;
    background-color: #f4f4f4;
    color: #333;
}

header {
    background-color: #333;
    color: #fff;
    padding: 1em 0;
    text-align: center;
}

.gallery-container {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 15px;
    padding: 20px;
```

```
.gallery-container {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 15px;
    padding: 20px;
    max-width: 1200px;
    margin: 20px auto;
}

.gallery-item {
    position: relative;
    overflow: hidden;
    border-radius: 8px;
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    transition: transform 0.3s ease-in-out;
}

.gallery-item:hover {
    transform: scale(1.05); /* Hover zoom effect */
}

.gallery-item img {
    width: 100%;
    height: 200px;
    object-fit: cover;
    display: block;
}
```

```
[12]
✓ 0s
.overlay {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.6);
    color: #fff;
    display: flex;
    justify-content: center;
    align-items: center;
    opacity: 0;
    transition: opacity 0.3s ease-in-out;
}

.gallery-item:hover .overlay {
    opacity: 1; /* Overlay effect on hover */
}

.overlay span {
    font-size: 1.2em;
    text-align: center;
    padding: 10px;
}

/* Mobile Responsiveness */
@media (max-width: 768px) {
    .gallery-container {
        grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

```
[12]
✓ 0s
        grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
        padding: 10px;
        gap: 10px;
    }

    .gallery-item img {
        height: 150px;
    }
}

@media (max-width: 480px) {
    .gallery-container {
        grid-template-columns: 1fr;
        padding: 5px;
    }

    .gallery-item {
        margin-bottom: 10px;
    }
}
"""

with open("style.css", "w") as f:
    f.write(css_code)

print("CSS code saved to style.css")

CSS code saved to style.css
```



Task 2 – Profile Card Generator

Create a profile card web page.

Requirements:

• Display user image, name, role, and social media links.

• Use AI to:

o Suggest modern UI colors and fonts.

o Design the card using Flexbox.

o Add hover animation and shadow effects

Profile Card HTML Structure

This HTML sets up the basic layout for the profile card, including placeholders for an image, name, role, and social media links.

[13]
0s

```
html_code = """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI-Generated Profile Card</title>
    <link rel="stylesheet" href="style.css">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min
</head>
<body>
    <div class="profile-card">
        <div class="profile-image">
            <img src="https://picsum.photos/id/64/200/200" alt="Profile Picture">
        </div>
        <div class="profile-info">
            <h2>Jane Doe</h2>
            <h3>Software Engineer</h3>
            <p>Passionate about coding, AI, and creating innovative solutions.</p>
        </div>
        <div class="social-links">
```

[13]
0s

```
        <div class="profile-info">
            <h2>Jane Doe</h2>
            <h3>Software Engineer</h3>
            <p>Passionate about coding, AI, and creating innovative solutions.</p>
        </div>
        <div class="social-links">
            <a href="#" class="social-icon"><i class="fab fa-github"></i></a>
            <a href="#" class="social-icon"><i class="fab fa-linkedin"></i></a>
            <a href="#" class="social-icon"><i class="fab fa-twitter"></i></a>
            <a href="#" class="social-icon"><i class="fab fa-instagram"></i></a>
        </div>
    </div>
</body>
</html>
"""

with open("index.html", "w") as f:
    f.write(html_code)

print("HTML code saved to index.html")
```

```
HTML code saved to index.html
```

Profile Card CSS Styling

This CSS provides modern styling for the profile card, utilizing Flexbox for layout, contemporary colors, Google Fonts, and hover effects with shadows for interactivity and visual appeal.

[14]
0s

```
css_code = """
@import url('https://fonts.googleapis.com/css2?family=Roboto:wght@300;400;700&display=swap');

body {
    font-family: 'Roboto', sans-serif;
    margin: 0;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
    background-color: #e0e0e0; /* Light gray background */
    color: #333;
}

.profile-card {
    background-color: #ffffff;
    border-radius: 15px;
    box-shadow: 0 10px 20px rgba(0, 0, 0, 0.15);
    padding: 30px;
    display: flex;
    flex-direction: column;
    align-items: center;
```

[14]
0s

```
.profile-card {
    background-color: #ffffff;
    border-radius: 15px;
    box-shadow: 0 10px 20px rgba(0, 0, 0, 0.15);
    padding: 30px;
    display: flex;
    flex-direction: column;
    align-items: center;
    max-width: 350px;
    text-align: center;
    transition: transform 0.3s ease, box-shadow 0.3s ease; /* Smooth transition for hover effects */
}

.profile-card:hover {
    transform: translateY(-5px) scale(1.02); /* Slight lift and zoom on hover */
    box-shadow: 0 15px 30px rgba(0, 0, 0, 0.25); /* Enhanced shadow on hover */
}

.profile-image img {
    width: 150px;
    height: 150px;
    border-radius: 50%;
    object-fit: cover;
    border: 5px solid #6c5ce7; /* Vibrant border color */
    margin-bottom: 20px;
}

.profile-info h2 {
    margin: 0 0 5px 0;
    color: #4a4a4a;
```

```
.profile-info h3 {
    margin: 0 0 15px 0;
    color: #8e8e8e;
    font-size: 1.2em;
    font-weight: 400;
}

.profile-info p {
    color: #666;
    font-size: 1em;
    line-height: 1.5;
    margin-bottom: 25px;
}

.social-links {
    display: flex;
    gap: 15px;
}

.social-icon {
    color: #6c5ce7; /* Matching vibrant color */
    font-size: 1.8em;
    transition: color 0.3s ease, transform 0.3s ease;
}

.social-icon:hover {
    color: #a29bfe; /* Lighter shade on hover */
    transform: scale(1.2); /* Icon grows on hover */
```

[14]
✓ 0s

```
/* Responsive adjustments */
@media (max-width: 480px) {
    .profile-card {
        margin: 20px;
        padding: 20px;
    }

    .profile-image img {
        width: 120px;
        height: 120px;
    }

    .profile-info h2 {
        font-size: 1.8em;
    }

    .profile-info h3 {
        font-size: 1em;
    }

    .social-icon {
        font-size: 1.5em;
    }
}
"""

with open("style.css", "w") as f:
```

```
[14]
0s
with open('style.css', 'w') as f:
    f.write(css_code)

print("CSS code saved to style.css")
```

```
CSS code saved to style.css
```

Instructions to View Your Profile Card Webpage

1. **Run the Python cells above:** This will create two files in your Colab environment: `index.html` and `style.css`.
2. **Download the files:**
   - In the Colab file browser (the folder icon on the left sidebar), locate `index.html` and `style.css`.
   - Right-click on each file and select 'Download'.
3. **Open `index.html`:** Once downloaded to your computer, open the `index.html` file with your web browser (e.g., Chrome, Firefox, Edge).

You should now see your profile card with its design, hover animations, and responsiveness!

Task 3 -Develop a dynamic shopping cart system.

Requirements:

• Add, remove, and update product quantities.

• Display real-time total price.

• Use AI to:

o Manage cart state using JavaScript objects.

o Persist cart data in localStorage.

Shopping Cart HTML Structure (index.html)

This HTML sets up the basic layout for the dynamic shopping cart, including product listings and a cart summary section. It links to our CSS and JavaScript files.

[15]
0s

```
html_code = """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic Shopping Cart</title>
    <link rel="stylesheet" href="style.css">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min
</head>
<body>
    <div class="container">
        <h1>My Awesome Shop</h1>

        <div class="products">
            <div class="product-item" data-id="1" data-name="Laptop" data-price="1200">
                <img src="https://picsum.photos/id/1005/200/200" alt="Laptop">
                <h2>Laptop</h2>
                <p>$1200.00</p>
                <button class="add-to-cart">Add to Cart</button>
            </div>
```

[15]
0s

```
                <button class="add-to-cart">Add to Cart</button>
            </div>
            <div class="product-item" data-id="2" data-name="Mouse" data-price="25">
                <img src="https://picsum.photos/id/49/200/200" alt="Mouse">
                <h2>Mouse</h2>
                <p>$25.00</p>
                <button class="add-to-cart">Add to Cart</button>
            </div>
            <div class="product-item" data-id="3" data-name="Keyboard" data-price="75">
                <img src="https://picsum.photos/id/292/200/200" alt="Keyboard">
                <h2>Keyboard</h2>
                <p>$75.00</p>
                <button class="add-to-cart">Add to Cart</button>
            </div>
        </div>

        <div class="cart-section">
            <h2>Your Shopping Cart</h2>
            <div class="cart-items">
                <!-- Cart items will be injected here by JavaScript -->
            </div>
            <div class="cart-summary">
                <p>Total: $<span id="cart-total">0.00</span></p>
                <button id="clear-cart">Clear Cart</button>
            </div>
        </div>
    </div>

    <script src="script.js"></script>
</body>
```

```
with open("index.html", "w") as f:
    f.write(html_code)

print("HTML code saved to index.html")
```

Shopping Cart CSS Styling (style.css)

This CSS provides a clean and modern design for the shopping cart, including responsive layout for products and cart items.

[16]
0s

```
css_code = """
body {
    font-family: 'Arial', sans-serif;
    margin: 0;
    background-color: #f8f8f8;
    color: #333;
    line-height: 1.6;
}

.container {
    max-width: 1200px;
    margin: 30px auto;
    padding: 20px;
    background-color: #fff;
    border-radius: 8px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}

h1, h2 {
    text-align: center;
    color: #3f51b5; /* Deep purple */
    margin-bottom: 25px;
```

[16]
0s

```
    margin-bottom: 25px;
}

.products {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 30px;
    margin-bottom: 40px;
}

.product-item {
    background-color: #fdfdfd;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
    text-align: center;
    transition: transform 0.2s ease-in-out;
}

.product-item:hover {
    transform: translateY(-5px);
}

.product-item img {
    max-width: 100%;
    height: 150px;
    object-fit: cover;
    border-radius: 4px;
    margin-bottom: 15px;
}
```

Commands + Code + Text Run all

[16]
0s

```
.product-item h2 {
    font-size: 1.5em;
    margin-top: 0;
    margin-bottom: 10px;
    color: #555;
}

.product-item p {
    font-size: 1.2em;
    color: #777;
    font-weight: bold;
    margin-bottom: 15px;
}

button {
    background-color: #4CAF50; /* Green */
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 1em;
    transition: background-color 0.2s ease;
}

button:hover {
    background-color: #45a049;
}
```

[16]
0s

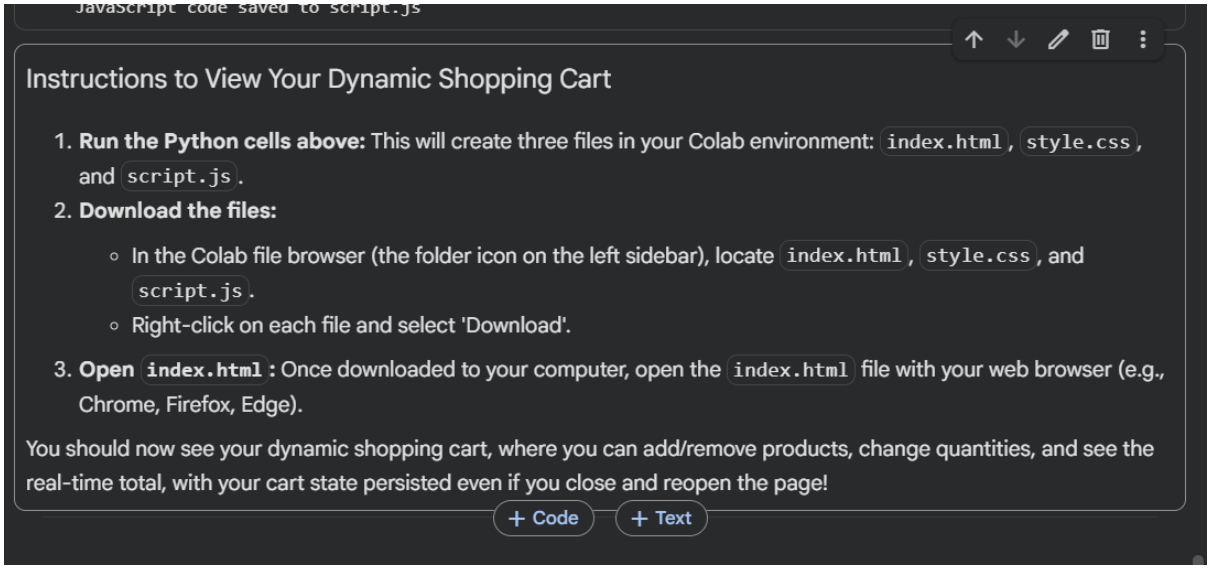
```
.cart-section {
    border-top: 2px solid #eee;
    padding-top: 30px;
}

.cart-items {
    margin-bottom: 20px;
}

.cart-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 15px 0;
    border-bottom: 1px dashed #eee;
}

.cart-item:last-child {
    border-bottom: none;
}

.cart-item-info {
    flex-grow: 1;
}

.cart-item-info h3 {
    margin: 0 0 5px 0;
    font-size: 1.2em;
    color: #444;
}
```

[16]
✓ 0s

```
    .cart-item-controls {
        display: flex;
        align-items: center;
        gap: 10px;
    }

    .quantity-btn {
        background-color: #f44336; /* Red */
        padding: 5px 10px;
        font-size: 1.1em;
    }

    .quantity-btn.add {
        background-color: #2196F3; /* Blue */
    }

    .remove-item {
        background-color: #f44336; /* Red */
        padding: 5px 10px;
        font-size: 0.9em;
    }

    #clear-cart {
        background-color: #f44336; /* Red */
        margin-top: 15px;
        padding: 12px 25px;
        font-size: 1.1em;
    }
```

```
}

/* Responsive adjustments */
@media (max-width: 768px) {
    .products {
        grid-template-columns: 1fr;
    }

    .cart-item {
        flex-direction: column;
        align-items: flex-start;
        gap: 10px;
    }

    .cart-item-controls {
        width: 100%;
        justify-content: flex-end;
    }
}
"""

with open("style.css", "w") as f:
    f.write(css_code)

print("CSS code saved to style.css")
```

```
CSS code saved to style.css
```



```
JavaScript code saved to script.js
```

Instructions to View Your Dynamic Shopping Cart

1. **Run the Python cells above:** This will create three files in your Colab environment: `index.html`, `style.css`, and `script.js`.
2. **Download the files:**
   - In the Colab file browser (the folder icon on the left sidebar), locate `index.html`, `style.css`, and `script.js`.
   - Right-click on each file and select 'Download'.
3. **Open `index.html`:** Once downloaded to your computer, open the `index.html` file with your web browser (e.g., Chrome, Firefox, Edge).

You should now see your dynamic shopping cart, where you can add/remove products, change quantities, and see the real-time total, with your cart state persisted even if you close and reopen the page!

+ Code + Text

Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

• Admin, Editor, User roles.

• Conditional UI rendering.

• Use AI to:

o Design access control logic.

o Maintain secure state transitions.

o Create scalable UI components.

Role-Based Access Control HTML Structure (index.html)

This HTML sets up the main page with a title, links to the CSS and JavaScript files, a header to display the current role, buttons to simulate role switching, and distinct sections for Admin, Editor, and User content.

[18]
0s

```
html_code = """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Role-Based Access Control</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <header>
            <h1>Access Control Application</h1>
            <p>Current Role: <span id="current-role">Guest</span></p>
            <div class="role-switcher">
                <button id="login-admin">Login as Admin</button>
                <button id="login-editor">Login as Editor</button>
                <button id="login-user">Login as User</button>
                <button id="logout">Logout</button>
            </div>
        </header>
```

[18]
0s

```
        <main>
            <section id="admin-content" class="content-section hidden">
                <h2>Admin Dashboard</h2>
                <p>Welcome, Admin! Here you can manage users and system settings.</p>
                <ul>
                    <li>Manage Users</li>
                    <li>View System Logs</li>
                    <li>Configure Settings</li>
                </ul>
            </section>

            <section id="editor-content" class="content-section hidden">
                <h2>Editor Panel</h2>
                <p>Welcome, Editor! You have access to content creation and modification.</p>
                <ul>
                    <li>Create New Article</li>
                    <li>Edit Existing Posts</li>
                    <li>Review Submissions</li>
                </ul>
            </section>

            <section id="user-content" class="content-section hidden">
                <h2>User Portal</h2>
                <p>Welcome, User! Access your personalized content and profile.</p>
                <ul>
                    <li>View My Profile</li>
                    <li>Read Articles</li>
                    <li>Submit Feedback</li>
                </ul>
```

[18]
0s

```
            <section id="user-content" class="content-section hidden">
                <h2>User Portal</h2>
                <p>Welcome, User! Access your personalized content and profile.</p>
                <ul>
                    <li>View My Profile</li>
                    <li>Read Articles</li>
                    <li>Submit Feedback</li>
                </ul>
            </section>

            <section id="guest-content" class="content-section">
                <h2>Guest View</h2>
                <p>Please log in to access more features.</p>
            </section>
        </main>
    </div>

    <script src="script.js"></script>
</body>
</html>
"""

with open("index.html", "w") as f:
    f.write(html_code)

print("HTML code saved to index.html")
```

Role-Based Access Control CSS Styling (style.css)

This CSS styles the application, providing a clean layout, modern colors, and hiding content sections by default. It also includes styling for buttons and basic responsiveness.

[19]
0s

```
css_code = """
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    margin: 0;
    background-color: #f4f7f6;
    color: #333;
    line-height: 1.6;
}

.container {
    max-width: 960px;
    margin: 30px auto;
    padding: 25px;
    background-color: #ffffff;
    border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
}

header {
    text-align: center;
    margin-bottom: 30px;
    padding-bottom: 20px;
```

[19]
0s

```
}

.content-section h2 {
    color: #2980b9;
    margin-top: 0;
}

.content-section ul {
    list-style-type: disc;
    padding-left: 20px;
}

.content-section li {
    margin-bottom: 8px;
}

.hidden {
    display: none;
}

/* Responsive adjustments */
@media (max-width: 600px) {
    .container {
        margin: 15px;
        padding: 15px;
    }

    .role-switcher button {
        width: 100%;
        margin-bottom: 5px;
    }
}
"""

with open("style.css", "w") as f:
    f.write(css_code)

print("CSS code saved to style.css")
```

```
CSS code saved to style.css
```

Role-Based Access Control JavaScript Logic (script.js)

This JavaScript manages the user roles, updates the UI based on the current role, and handles the simulated login/logout actions. It ensures only relevant content is displayed, demonstrating secure state transitions and conditional rendering.

[20]
0s

```
js_code = """
document.addEventListener('DOMContentLoaded', () => {
    const currentRoleSpan = document.getElementById('current-role');
    const adminBtn = document.getElementById('login-admin');
    const editorBtn = document.getElementById('login-editor');
    const userBtn = document.getElementById('login-user');
    const logoutBtn = document.getElementById('logout');

    const adminContent = document.getElementById('admin-content');
    const editorContent = document.getElementById('editor-content');
    const userContent = document.getElementById('user-content');
    const guestContent = document.getElementById('guest-content');

    let userRole = localStorage.getItem('userRole') || 'guest'; // Persist role

    const updateUI = () => {
        // Hide all content sections first
        adminContent.classList.add('hidden');
        editorContent.classList.add('hidden');
        userContent.classList.add('hidden');
        guestContent.classList.add('hidden');

        currentRoleSpan.textContent = userRole.charAt(0).toUpperCase() + userRole.slice(1); // Capitalize first letter

        // Show content based on role
        switch (userRole) {
            case 'admin':
                adminContent.classList.remove('hidden');
                break;
            case 'editor':
                editorContent.classList.remove('hidden');
                break;
            case 'user':
                userContent.classList.remove('hidden');
                break;
            default:
                guestContent.classList.remove('hidden');
                break;
        }

        localStorage.setItem('userRole', userRole); // Save current role
    };

    const setRole = (role) => {
        userRole = role;
        updateUI();
```

Variables Terminal

```
[20]
✓ 0s
                default:
                    guestContent.classList.remove('hidden');
                    break;
            }

            localStorage.setItem('userRole', userRole); // Save current role
        };

        const setRole = (role) => {
            userRole = role;
            updateUI();
        };

        adminBtn.addEventListener('click', () => setRole('admin'));
        editorBtn.addEventListener('click', () => setRole('editor'));
        userBtn.addEventListener('click', () => setRole('user'));
        logoutBtn.addEventListener('click', () => setRole('guest'));

        // Initial UI update on page load
        updateUI();
    });
    """

    with open("script.js", "w") as f:
        f.write(js_code)

    print("JavaScript code saved to script.js")
```

```
JavaScript code saved to script.js
```

Instructions to View Your Role-Based Access Control Application

1. **Run the Python cells above:** This will create three files in your Colab environment: `index.html`, `style.css`, and `script.js`.
2. **Download the files:**
   - In the Colab file browser (the folder icon on the left sidebar), locate `index.html`, `style.css`, and `script.js`.
   - Right-click on each file and select 'Download'.
3. **Open `index.html`:** Once downloaded to your computer, open the `index.html` file with your web browser (e.g., Chrome, Firefox, Edge).

You should now see the application. Click on the different 'Login as...' buttons to observe how the content changes based on the assigned role, demonstrating conditional UI rendering and secure state transitions!

Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

• Switch languages dynamically.

• Persist language preference.

• Use AI to:

o Structure translation files.

o Handle RTL/LTR layouts.

o Improve accessibility.

This HTML sets up the basic layout of the multi-language website, including content sections with data attributes for translation keys, and buttons for language switching. It links to the CSS and JavaScript files.

[21]
0s

```
html_code = """
<!DOCTYPE html>
<html lang="en" dir="ltr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title data-translate="page_title">Multi-Language Website</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <header>
            <h1 data-translate="welcome_heading">Welcome to Our Website!</h1>
            <div class="lang-switcher">
                <button data-lang="en" aria-label="Switch to English">English</button>
                <button data-lang="es" aria-label="Cambiar a español">Español</button>
                <button data-lang="ar" aria-label="التبديل إلى العربية">العربية</button>
            </div>
        </header>

        <main>
            <section>
                <h2 data-translate="about_us_title">About Us</h2>
                <p data-translate="about_us_content">We are a company dedicated to providing excellent services and products to our customers worldwide.</p>
            </section>

            <section>
                <h2 data-translate="services_title">Our Services</h2>
                <ul>
                    <li data-translate="service_1">Web Development</li>
                    <li data-translate="service_2">Mobile App Development</li>
                    <li data-translate="service_3">UI/UX Design</li>
                </ul>
            </section>

            <section>
                <h2 data-translate="contact_title">Contact Us</h2>
                <p data-translate="contact_content">Feel free to reach out to us for any inquiries or support.</p>
            </section>
        </main>

        <footer>
            <p data-translate="footer_text">© 2023 Multi-Language Website. All rights reserved.</p>
        </footer>
    </div>

    <script src="script.js"></script>
</body>
```

Multi-Language Website CSS Styling (style.css)

This CSS provides a clean and responsive design for the website, including basic layout and styling for text and buttons. It also includes styling for `rtl` (right-to-left) direction for languages like Arabic.

[22]
0s

```
css_code = """
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    margin: 0;
    background-color: #f4f7f6;
    color: #333;
    line-height: 1.6;
}

.container {
    max-width: 960px;
    margin: 30px auto;
    padding: 25px;
    background-color: #ffffff;
    border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
}

header {
    text-align: center;
    margin-bottom: 30px;
    padding-bottom: 20px;
    border-bottom: 1px solid #eee;
}

header h1 {
    color: #2c3e50;
    margin-bottom: 10px;
}

.lang-switcher {
    margin-top: 20px;
    display: flex;
    justify-content: center;
    gap: 10px;
}

.lang-switcher button {
    background-color: #3498db;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 0.9em;
```

[22]
✓ 0s

```
section {
    padding: 20px;
    margin-top: 20px;
    border: 1px dashed #dcdcdc;
    border-radius: 8px;
    background-color: #fdfdfd;
}

section h2 {
    color: #2980b9;
    margin-top: 0;
}

section ul {
    list-style-type: disc;
    padding-left: 20px;
}

section li {
    margin-bottom: 8px;
}

footer {
    text-align: center;
    margin-top: 30px;
    padding-top: 20px;
    border-top: 1px solid #eee;
    color: #777;
    font-size: 0.9em;
}

/* RTL specific styles */
html[dir="rtl"] body {
    direction: rtl;
    text-align: right;
}

html[dir="rtl"] section ul {
    padding-right: 20px;
    padding-left: 0;
    list-style-type: arabic-indic;
}
"""

with open("style.css", "w") as f:
    f.write(css_code)

print("CSS code saved to style.css")
```

```
CSS code saved to style.css
```

Multi-Language Website JavaScript Logic (script.js)

This JavaScript manages the translation data, dynamically switches the language of the page content, persists the user's language preference using `localStorage`, and handles RTL/LTR layout changes for improved accessibility and user experience.

[23]
✓ 0s

```
js_code = """
document.addEventListener('DOMContentLoaded', () => {
    const langButtons = document.querySelectorAll('.lang-switcher button');
    const htmlElement = document.querySelector('html');

    // Translation data (structured for easy management)
    const translations = {
        en: {
            page_title: "Multi-Language Website",
            welcome_heading: "Welcome to Our Website!",
            about_us_title: "About Us",
            about_us_content: "We are a company dedicated to providing excellent services and products to our customers worldwide.",
            services_title: "Our Services",
            service_1: "Web Development",
            service_2: "Mobile App Development",
            service_3: "UI/UX Design",
            contact_title: "Contact Us",
            contact_content: "Feel free to reach out to us for any inquiries or support.",
            footer_text: "© 2023 Multi-Language Website. All rights reserved."
        },
        es: {
            page_title: "Sitio Web Multilingüe",
            welcome_heading: "¡Bienvenido a Nuestro Sitio Web!",
            about_us_title: "Sobre Nosotros",
            about_us_content: "Somos una empresa dedicada a proporcionar excelentes servicios y productos a nuestros clientes en todo el mundo.",
            services_title: "Nuestros Servicios",
            service_1: "Desarrollo Web",
            service_2: "Desarrollo de Aplicaciones Móviles",
            service_3: "Diseño UI/UX",
            contact_title: "Contáctanos",
            contact_content: "No dudes en contactarnos para cualquier consulta o soporte.",
            footer_text: "© 2023 Sitio Web Multilingüe. Todos los derechos reservados."
        },
        ar: {
            page_title: "موقع متعدد اللغات",
            welcome_heading: "!مرحباً بكم في موقعنا",
            about_us_title: "من نحن",
            about_us_content: "نحن شركة متخصصة في تقديم خدمات ومنتجات ممتازة لعملائنا في جميع أنحاء العالم.",
            services_title: "خدماتنا",
            service_1: "تطوير الويب",
            service_2: "تطوير تطبيقات الجوال",
            service_3: "تصميم واجهة المستخدم/تجربة المستخدم",
            contact_title: "اتصل بنا",
            contact_content: "لا تتردد في التواصل معنا لأي استفسارات أو دعم.",
            footer_text: "© 2023 موقع متعدد اللغات. جميع الحقوق محفوظة."
        }
```

```
[23]
0s
    };

    // Function to update the content based on the selected language
    const applyTranslations = (lang) => {
        document.querySelectorAll('[data-translate]').forEach(element => {
            const key = element.dataset.translate;
            if (translations[lang] && translations[lang][key]) {
                element.textContent = translations[lang][key];
            }
        });
        // Update document title separately as it's not directly in body
        document.title = translations[lang].page_title;
        setDirection(lang); // Apply text direction
        localStorage.setItem('selectedLanguage', lang); // Persist preference
    };

    // Event listeners for language buttons
    langButtons.forEach(button => {
        button.addEventListener('click', () => {
            const lang = button.dataset.lang;
            applyTranslations(lang);
        });
    });

    // Load preferred language from localStorage or default to English
    const storedLang = localStorage.getItem('selectedLanguage') || 'en';
    applyTranslations(storedLang);
});
"""

with open("script.js", "w") as f:
    f.write(js_code)

print("JavaScript code saved to script.js")
```

```
JavaScript code saved to script.js
```

Instructions to View Your Multi-Language Website

1. **Run the Python cells above:** This will create three files in your Colab environment: `index.html`, `style.css`, and `script.js`.
2. **Download the files:**
   - In the Colab file browser (the folder icon on the left sidebar), locate `index.html`, `style.css`, and `script.js`.
   - Right-click on each file and select 'Download'.
3. **Open `index.html`:** Once downloaded to your computer, open the `index.html` file with your web browser (e.g., Chrome, Firefox, Edge).

You should now see your multi-language website. Click on the language buttons to switch languages, and observe how the text changes and how the layout adjusts for Right-to-Left (RTL) languages like Arabic!

Task 6 – Online Examination System

Create an online exam interface.

Requirements:

• Timer, questions, and submission.

• Prevent multiple submissions.

• Use AI to:

o Manage exam state.

o Improve exam UX.

o Ensure accessibility.

Online Examination System HTML Structure (index.html)

This HTML sets up the basic layout for the online exam, including a header with a timer, a main section for questions and navigation, and a submission button. It links to the CSS and JavaScript files.

[24]
0s

```
html_code = """
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Examination</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <header>
            <h1>Online Examination System</h1>
            <div class="timer" aria-live="polite">Time Left: <span id="time">--:--</span></div>
        </header>

        <main>
            <div id="quiz-area">
                <div id="question-container" class="question-container">
                    <h2 id="question-text">Question goes here...</h2>
                    <div id="answer-options" class="answer-options">
                        <!-- Answer options will be dynamically loaded here -->
                    </div>
                </div>

                <div class="navigation-controls">
                    <button id="prev-btn" class="nav-btn">Previous</button>
                    <button id="next-btn" class="nav-btn">Next</button>
                </div>

                <button id="submit-btn" class="submit-btn" disabled>Submit Exam</button>
            </div>

            <div id="result-area" class="result-area hidden">
                <h2>Exam Results</h2>
                <p>You scored: <span id="score"></span> / <span id="total-questions"></span></p>
                <button id="retake-btn">Retake Exam</button>
            </div>

            <div id="submission-message" class="submission-message hidden" aria-live="assertive">
                <!-- Submission messages will appear here -->
            </div>
        </main>
    </div>
```

Online Examination System CSS Styling (style.css)

This CSS provides a clean and focused design for the examination interface, including styling for questions, answer options, timer, and navigation buttons. It also handles the hidden state for results and submission messages.

[25]
0s

```
css_code = """
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    margin: 0;
    background-color: #f4f7f6;
    color: #333;
    line-height: 1.6;
}

.container {
    max-width: 800px;
    margin: 30px auto;
    padding: 25px;
    background-color: #ffffff;
    border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
}

header {
    text-align: center;
    margin-bottom: 30px;
    padding-bottom: 20px;
    border-bottom: 1px solid #eee;
}

header h1 {
    color: #2c3e50;
    margin-bottom: 10px;
}

.timer {
    font-size: 1.2em;
    font-weight: bold;
    color: #e74c3c; /* Red for urgency */
}

.question-container {
    margin-bottom: 25px;
    padding: 20px;
    border: 1px solid #ddd;
    border-radius: 8px;
    background-color: #fdfdfd;
}
```

```
[25]
✓ 0s
        margin-bottom: 10px;
        padding: 10px;
        background-color: #ecf0f1; /* Light gray */
        border-radius: 5px;
        cursor: pointer;
        transition: background-color 0.2s ease;
    }

    .answer-options label:hover {
        background-color: #e0e6e7;
    }

    .answer-options input[type="radio"] {
        margin-right: 10px;
    }

    .answer-options input[type="radio"]:checked + span {
        font-weight: bold;
        color: #2980b9;
    }

    .navigation-controls {
        display: flex;
        justify-content: space-between;
        margin-top: 20px;
    }

    .nav-btn, .submit-btn, #retake-btn {
        background-color: #2ecc71; /* Green */
        color: white;
        padding: 12px 25px;
        border: none;
        border-radius: 5px;
        cursor: pointer;
        font-size: 1em;
        transition: background-color 0.2s ease, transform 0.1s ease;
    }

    .nav-btn:hover, #retake-btn:hover {
        background-color: #27ae60;
        transform: translateY(-2px);
    }

    .nav-btn:disabled, .submit-btn:disabled {
        background-color: #bdc3c7; /* Gray for disabled */
        cursor: not-allowed;
        transform: none;
    }

    .submit-btn {
        display: block;
        width: 100%;
        margin-top: 30px;
```

Commands + Code + Text Run all

```
[25]
✓ 0s
    }

    .result-area h2 {
        color: #155724;
    }

    #retake-btn {
        margin-top: 20px;
        background-color: #3498db; /* Blue for retake */
    }

    #retake-btn:hover {
        background-color: #2980b9;
    }

    .submission-message {
        margin-top: 20px;
        padding: 15px;
        border-radius: 8px;
        text-align: center;
        font-weight: bold;
    }

    .submission-message.success {
        background-color: #d4edda;
        color: #155724;
    }

    .submission-message.error {
        background-color: #f8d7da;
        color: #721c24;
    }

    .hidden {
        display: none;
    }

    /* Accessibility: Focus styles */
    button:focus, label:focus-within {
        outline: 2px solid #3498db;
        outline-offset: 2px;
    }
    """

    with open("style.css", "w") as f:
        f.write(css_code)

    print("CSS code saved to style.css")

CSS code saved to style.css
```

Online Examination System JavaScript Logic (script.js)

This JavaScript manages the exam flow: displaying questions, starting and stopping a timer, recording user answers, calculating results, and preventing multiple submissions using `localStorage` to manage the exam state. It also includes basic UX enhancements.

[26]
0s

```
js_code = """
document.addEventListener('DOMContentLoaded', () => {
    const questions = [
        {
            question: "What is the capital of France?",
            options: ["Berlin", "Madrid", "Paris", "Rome"],
            answer: "Paris"
        },
        {
            question: "Which planet is known as the Red Planet?",
            options: ["Earth", "Mars", "Jupiter", "Venus"],
            answer: "Mars"
        },
        {
            question: "What is 7 multiplied by 8?",
            options: ["49", "56", "64", "72"],
            answer: "56"
        },
        {
            question: "Who painted the Mona Lisa?",
            options: ["Vincent van Gogh", "Pablo Picasso", "Leonardo da Vinci", "Claude Monet"],
            answer: "Leonardo da Vinci"
        }
    ];

    const quizArea = document.getElementById('quiz-area');
    const resultArea = document.getElementById('result-area');
    const submissionMessage = document.getElementById('submission-message');
    const questionText = document.getElementById('question-text');
    const answerOptionsDiv = document.getElementById('answer-options');
    const timeSpan = document.getElementById('time');
    const prevBtn = document.getElementById('prev-btn');
    const nextBtn = document.getElementById('next-btn');
    const submitBtn = document.getElementById('submit-btn');
    const scoreSpan = document.getElementById('score');
    const totalQuestionsSpan = document.getElementById('total-questions');
    const retakeBtn = document.getElementById('retake-btn');

    let currentQuestionIndex = 0;
    let userAnswers = {}; // Store answers as {questionIndex: selectedOption}
    let timeLeft = 60 * 2; // 2 minutes in seconds
    let timerInterval;
    let examSubmitted = false;

    // --- Local Storage Management ---
    const loadExamState = () => {
```

Variables Terminal

ommands + Code + Text Run all

[26]
0s

```
            displaySubmissionMessage('Time is up! Your exam has been automatically submitted.', 'error');
            submitExam(); // Auto-submit if time ran out
            return true;
        }
    }
    return false;
};

const saveExamState = () => {
    const stateToSave = {
        userAnswers,
        currentQuestionIndex,
        timeLeft,
        examSubmitted
    };
    localStorage.setItem('examState', JSON.stringify(stateToSave));
};

const clearExamState = () => {
    localStorage.removeItem('examState');
    userAnswers = {};
    currentQuestionIndex = 0;
    timeLeft = 60 * 2; // Reset timer for new exam
    examSubmitted = false;
};

// --- Timer Functions ---
const startTimer = () => {
    timerInterval = setInterval(() => {
        if (examSubmitted) {
            stopTimer();
            return;
        }
        timeLeft--;
        const minutes = Math.floor(timeLeft / 60);
        const seconds = timeLeft % 60;
        timeSpan.textContent = `${minutes.toString().padStart(2, '0')}:${seconds.toString().padStart(2, '0')}`;

        if (timeLeft <= 0) {
            stopTimer();
            submitExam();
            displaySubmissionMessage('Time is up! Your exam has been automatically submitted.', 'error');
        }
        saveExamState();
    }, 1000);
};

const stopTimer = () => {
    clearInterval(timerInterval);
};

// --- Question Rendering ---
const renderQuestion = () => {
```

```
    });

    submitBtn.addEventListener('click', () => {
        if (confirm('Are you sure you want to submit your exam?')) {
            submitExam();
        }
    });

    retakeBtn.addEventListener('click', () => {
        clearExamState();
        resultArea.classList.add('hidden');
        submissionMessage.classList.add('hidden');
        quizArea.classList.remove('hidden');
        currentQuestionIndex = 0;
        timeLeft = 60 * 2;
        startTimer();
        renderQuestion();
    });

    // --- Initialize Exam ---
    if (!loadExamState()) {
        startTimer();
        renderQuestion();
        totalQuestionsSpan.textContent = questions.length;
    }
});
"""

with open("script.js", "w") as f:
    f.write(js_code)

print("JavaScript code saved to script.js")
```

```
JavaScript code saved to script.js
```

Instructions to View Your Online Examination System

1. **Run the Python cells above:** This will create three files in your Colab environment: `index.html`, `style.css`, and `script.js`.
2. **Download the files:**
   - In the Colab file browser (the folder icon on the left sidebar), locate `index.html`, `style.css`, and `script.js`.
   - Right-click on each file and select 'Download'.
3. **Open `index.html`:** Once downloaded to your computer, open the `index.html` file with your web browser (e.g., Chrome, Firefox, Edge).

You should now see your online examination system. Attempt the questions, use the navigation, and observe the timer. Try submitting and then refreshing the page to see how it prevents multiple submissions and retains your state!

Terminal

Search